تطبيق جملة مع التجميع يعني يكون فيه تجار جملة والعميلاء يطلبو ويدفعو وبعد ما يكتمل عدد معين (حملات التاجر يقول مثلا حملة ١٠٠ منتج بحسب المنتج ب ٢٠ ريال بدلا من ٥٠ ، العملا يشاركو واول ما يكتمل العدد يتم ارسال الطلب للتاجر ويتم التوصيل) المنصة عبارة عن ٣ تطبيقات تطبيق تاجر ومندوب وعميل ، العناوين مبنية على اختيار المنطقة والمدينة والحي وكتابة رقم الهاتف واسم العنوان

فهمت الفكرة. أنت تريد **منصة شراء جماعي من تجار الجملة**، مكوّنة من 3 تطبيقات مترابطة:

1. تطبيق العميل

- تسجيل الدخول برقم الجوال.
- اختيار **المنطقة** ← **المدينة** ← **الحي**.
- إنشاء عناوين محفوظة، وكل عنوان يحتوي:
  - اسم العنوان
  - المنطقة
  - المدينة
  - الحي
  - رقم الهاتف
- تصفح حملات التجار.
- مثال: **حملة 100 قطعة**
  - سعر التجزئة: 50 ريال
  - سعر الحملة: 20 ريال
  - المطلوب: 100 قطعة
  - المشاركون الحاليون: 73/100
- العميل يختار الكمية ويدفع مباشرة.
- عندما يصل العدد إلى 100:
  - تُغلق الحملة.
  - يُرسل الطلب المجمع للتاجر.
  - التاجر يجهز الشحنة.
  - يتم إسناد التوصيل لمندوب.
  - العميل يتابع حالة طلبه.

2. تطبيق التاجر

التاجر يستطيع:

- إنشاء حملات جديدة.
- تحديد المنتج والصور والوصف.
- تحديد **الحد الأدنى للكمية** للحملة.
- تحديد سعر الحملة.
- تحديد مدة الحملة.
- تحديد الكمية المتاحة.
- مشاهدة عدد المشاركين والطلبات المدفوعة.
- عند اكتمال الحملة يستلم **طلبًا مجمعًا**.
- تجهيز الطلب وتسليمه للمندوب.

- متابعة المبيعات والحملات والأرباح.

3. تطبيق المندوب

- استقبال طلبات التوصيل.
- مشاهدة عنوان العميل.
- استلام الطلب من التاجر.
- تحديث الحالة:

  تم الاستلام → في الطريق → تم التسليم
- إمكانية التواصل مع العميل.
- سجل الطلبات والتوصيلات والدخل.

دورة الطلب الأساسية

التاجر ينشئ حملة

↓

العملاء يشتركون ويدفعون

↓

العدد يصل إلى الحد المطلوب

↓

الحملة تُغلق تلقائيًا

↓

إرسال الطلب المجمع للتاجر

↓

التاجر يجهز المنتجات

↓

المندوب يستلمها

↓

التوصيل للعملاء

↓

إغلاق الطلب

والأهم أن النظام يحتاج من البداية إلى معالجة حالات مثل **إلغاء العميل، فشل الدفع، عدم اكتمال الحملة، نفاد المنتج، الاسترجاع، والطلبات الجزئية**.

إذا كان هدفك الآن **بناء المنصة فعليًا**، أقدر أرتب لك المشروع كاملًا كـ **مواصفات جاهزة للمبرمج** تشمل الشاشات، قاعدة البيانات، الصلاحيات، حالات الطلب والحملة، ونظام الدفع والتوصيل، ثم نبدأ في تصميم الواجهات.

لمنصة البيع بالجملة والتجميع، خيارات أدوات تسويق مدعومة بالذكاء الاصطناعي قد تساعد.



Mudrek LLC

٢٩ أداة ذكاء اصطناعي لصاحب المشروع

إعلانات، صور منتجات، شعارات، فيديو ونصوص تسويقية. كلها بالعربي في مكان واحد. جرّب بـ ٣$

إعلان

المزيد من الخيارات

اكتب ال model c# مع العلاقات اكتبها في بان واحد حتي اخدهم كوبي ، استخدم identity for authorization

أكيد. هذا تصميم مبدئي لـ **Entity Framework Core + ASP.NET Core Identity** لمنصة الشراء الجماعي، وكل الـ Models في **ملف واحد** حتى تستطيع نسخها مباشرة.

اعتمدت على:

- `IdentityUser<Guid>` للمستخدمين والصلاحيات.
- 3 أنواع مستخدمين: **Customer / Merchant / Driver** باستخدام Roles.
- المناطق: `Region → City → District`.
- العناوين مرتبطة بالعميل.
- التاجر ينشئ Campaign.
- العميل يشارك في الحملة من خلال `CampaignOrder`.
- عند اكتمال الحملة يتم إنشاء `MerchantOrder`.
- التوصيل من خلال `Delivery`.
- دعم الدفع وحالات الطلب والحملة.
- كل العلاقات الأساسية موجودة.

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace WholesaleGroupBuying.Models
{
    //
================================================================
    // IDENTITY USER
    //
================================================================

    public class ApplicationUser : IdentityUser<Guid>
    {
        [MaxLength(150)]
        public string FullName { get; set; } = string.Empty;

        public UserType UserType { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
```

```
        // Navigation
        public CustomerProfile? CustomerProfile { get; set; }
        public MerchantProfile? MerchantProfile { get; set; }
        public DriverProfile? DriverProfile { get; set; }

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();

        public ICollection<Campaign> Campaigns { get; set; }
            = new List<Campaign>();

        public ICollection<CampaignOrder> CampaignOrders
{ get; set; }
            = new List<CampaignOrder>();

        public ICollection<Delivery> Deliveries { get; set; }
            = new List<Delivery>();
    }


    //
============================================================
    // CUSTOMER
    //
============================================================

    public class CustomerProfile
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid UserId { get; set; }

        [ForeignKey(nameof(UserId))]
        public ApplicationUser User { get; set; } = null!;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
```

```
    // MERCHANT
    //
============================================================

    public class MerchantProfile
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid UserId { get; set; }

        [ForeignKey(nameof(UserId))]
        public ApplicationUser User { get; set; } = null!;

        [Required]
        [MaxLength(200)]
        public string BusinessName { get; set; } =
string.Empty;

        [MaxLength(100)]
        public string? CommercialRegistrationNumber { get;
set; }

        [MaxLength(500)]
        public string? LogoUrl { get; set; }

        [MaxLength(1000)]
        public string? Description { get; set; }

        public bool IsVerified { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        // Navigation
        public ICollection<Campaign> Campaigns { get; set; }
            = new List<Campaign>();
    }


    //
============================================================
    // DRIVER
```

```
    //
============================================================

    public class DriverProfile
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid UserId { get; set; }

        [ForeignKey(nameof(UserId))]
        public ApplicationUser User { get; set; } = null!;

        [MaxLength(50)]
        public string? NationalId { get; set; }

        [MaxLength(50)]
        public string? DrivingLicenseNumber { get; set; }

        [MaxLength(100)]
        public string? VehicleType { get; set; }

        [MaxLength(50)]
        public string? VehiclePlateNumber { get; set; }

        public bool IsAvailable { get; set; } = true;

        public bool IsVerified { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
    // REGION
    //
============================================================

    public class Region
    {
        [Key]
        public int Id { get; set; }
```

```
        [Required]
        [MaxLength(100)]
        public string NameAr { get; set; } = string.Empty;

        [MaxLength(100)]
        public string? NameEn { get; set; }

        public bool IsActive { get; set; } = true;

        public ICollection<City> Cities { get; set; }
            = new List<City>();

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();
    }


    //
============================================================
    // CITY
    //
============================================================

    public class City
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public int RegionId { get; set; }

        [ForeignKey(nameof(RegionId))]
        public Region Region { get; set; } = null!;

        [Required]
        [MaxLength(100)]
        public string NameAr { get; set; } = string.Empty;

        [MaxLength(100)]
        public string? NameEn { get; set; }

        public bool IsActive { get; set; } = true;

        public ICollection<District> Districts { get; set; }
```

```
            = new List<District>();

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();
    }


    //
============================================================
    // DISTRICT
    //
============================================================

    public class District
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public int CityId { get; set; }

        [ForeignKey(nameof(CityId))]
        public City City { get; set; } = null!;

        [Required]
        [MaxLength(100)]
        public string NameAr { get; set; } = string.Empty;

        [MaxLength(100)]
        public string? NameEn { get; set; }

        public bool IsActive { get; set; } = true;

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();
    }


    //
============================================================
    // CUSTOMER ADDRESS
    //
============================================================

    public class Address
```

```
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid UserId { get; set; }

        [ForeignKey(nameof(UserId))]
        public ApplicationUser User { get; set; } = null!;

        [Required]
        public int RegionId { get; set; }

        [ForeignKey(nameof(RegionId))]
        public Region Region { get; set; } = null!;

        [Required]
        public int CityId { get; set; }

        [ForeignKey(nameof(CityId))]
        public City City { get; set; } = null!;

        [Required]
        public int DistrictId { get; set; }

        [ForeignKey(nameof(DistrictId))]
        public District District { get; set; } = null!;

        [Required]
        [MaxLength(100)]
        public string AddressName { get; set; } =
string.Empty;

        [Required]
        [MaxLength(20)]
        public string PhoneNumber { get; set; } =
string.Empty;

        [MaxLength(500)]
        public string? AdditionalDetails { get; set; }

        public bool IsDefault { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
```

```
    }


    //
============================================================
    // PRODUCT
    //
============================================================

    public class Product
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid MerchantId { get; set; }

        [ForeignKey(nameof(MerchantId))]
        public MerchantProfile Merchant { get; set; } =
null!;

        [Required]
        [MaxLength(200)]
        public string NameAr { get; set; } = string.Empty;

        [MaxLength(200)]
        public string? NameEn { get; set; }

        [MaxLength(2000)]
        public string? Description { get; set; }

        [MaxLength(500)]
        public string? MainImageUrl { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<ProductImage> Images { get; set; }
            = new List<ProductImage>();

        public ICollection<Campaign> Campaigns { get; set; }
            = new List<Campaign>();
    }
```

```
    //
============================================================
    // PRODUCT IMAGE
    //
============================================================

    public class ProductImage
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid ProductId { get; set; }

        [ForeignKey(nameof(ProductId))]
        public Product Product { get; set; } = null!;

        [Required]
        [MaxLength(500)]
        public string ImageUrl { get; set; } = string.Empty;

        public int SortOrder { get; set; }
    }


    //
============================================================
    // GROUP BUYING CAMPAIGN
    //
============================================================

    public class Campaign
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid MerchantId { get; set; }

        [ForeignKey(nameof(MerchantId))]
        public MerchantProfile Merchant { get; set; } =
null!;
```

```
        [Required]
        public Guid ProductId { get; set; }

        [ForeignKey(nameof(ProductId))]
        public Product Product { get; set; } = null!;

        [Required]
        [MaxLength(200)]
        public string Title { get; set; } = string.Empty;

        [MaxLength(2000)]
        public string? Description { get; set; }

        // السعر الطبيعي
        [Column(TypeName = "decimal(18,2)")]
        public decimal OriginalPrice { get; set; }

        // سعر الحملة
        [Column(TypeName = "decimal(18,2)")]
        public decimal CampaignPrice { get; set; }

        // العدد المطلوب لإكمال الحملة
        public int RequiredQuantity { get; set; }

        // العدد الحالي
        public int CurrentQuantity { get; set; }

        // الحد الأقصى لكل عميل
        public int? MaxQuantityPerCustomer { get; set; }

        // مدة الحملة
        public DateTime StartDate { get; set; }

        public DateTime EndDate { get; set; }

        public CampaignStatus Status { get; set; }

        public DateTime? CompletedAt { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        // Navigation
        public ICollection<CampaignOrder> Orders { get;
set; }
```

```
            = new List<CampaignOrder>();

        public MerchantOrder? MerchantOrder { get; set; }
    }


    //
============================================================
    // CUSTOMER PARTICIPATION / ORDER
    //
============================================================

    public class CampaignOrder
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid CampaignId { get; set; }

        [ForeignKey(nameof(CampaignId))]
        public Campaign Campaign { get; set; } = null!;

        [Required]
        public Guid CustomerId { get; set; }

        [ForeignKey(nameof(CustomerId))]
        public ApplicationUser Customer { get; set; } =
null!;

        // عنوان التوصيل وقت الطلب
        [Required]
        public Guid AddressId { get; set; }

        [ForeignKey(nameof(AddressId))]
        public Address Address { get; set; } = null!;

        public int Quantity { get; set; }

        [Column(TypeName = "decimal(18,2)")]
        public decimal UnitPrice { get; set; }

        [Column(TypeName = "decimal(18,2)")]
        public decimal TotalAmount { get; set; }
```

```
        public OrderStatus Status { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        // Payment
        public Payment? Payment { get; set; }

        // Delivery
        public Delivery? Delivery { get; set; }
    }


    //
============================================================
    // PAYMENT
    //
============================================================

    public class Payment
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid CampaignOrderId { get; set; }

        [ForeignKey(nameof(CampaignOrderId))]
        public CampaignOrder CampaignOrder { get; set; } =
null!;

        [Column(TypeName = "decimal(18,2)")]
        public decimal Amount { get; set; }

        [Required]
        [MaxLength(50)]
        public string PaymentMethod { get; set; } =
string.Empty;

        [MaxLength(200)]
        public string? TransactionId { get; set; }

        public PaymentStatus Status { get; set; }

        public DateTime? PaidAt { get; set; }
```

```
        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
    // MERCHANT ORDER
    //
============================================================
    // الطلب المجمع الذي يصل للتاجر بعد اكتمال الحملة

    public class MerchantOrder
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid CampaignId { get; set; }

        [ForeignKey(nameof(CampaignId))]
        public Campaign Campaign { get; set; } = null!;

        [Required]
        public Guid MerchantId { get; set; }

        [ForeignKey(nameof(MerchantId))]
        public MerchantProfile Merchant { get; set; } =
null!;

        public int TotalQuantity { get; set; }

        [Column(TypeName = "decimal(18,2)")]
        public decimal TotalAmount { get; set; }

        public MerchantOrderStatus Status { get; set; }

        public DateTime? PreparedAt { get; set; }

        public DateTime? HandedToDriverAt { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
```

```
        public ICollection<Delivery> Deliveries { get; set; }
            = new List<Delivery>();
    }


    //
============================================================
    // DELIVERY
    //
============================================================

    public class Delivery
    {
        [Key]
        public Guid Id { get; set; }

        [Required]
        public Guid CampaignOrderId { get; set; }

        [ForeignKey(nameof(CampaignOrderId))]
        public CampaignOrder CampaignOrder { get; set; } =
null!;

        [Required]
        public Guid MerchantOrderId { get; set; }

        [ForeignKey(nameof(MerchantOrderId))]
        public MerchantOrder MerchantOrder { get; set; } =
null!;

        public Guid? DriverId { get; set; }

        [ForeignKey(nameof(DriverId))]
        public ApplicationUser? Driver { get; set; }

        public DeliveryStatus Status { get; set; }

        public DateTime? AssignedAt { get; set; }

        public DateTime? PickedUpAt { get; set; }

        public DateTime? OutForDeliveryAt { get; set; }

        public DateTime? DeliveredAt { get; set; }
```

```
        [MaxLength(1000)]
        public string? DriverNotes { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
    // ENUMS
    //
============================================================

    public enum UserType
    {
        Customer = 1,
        Merchant = 2,
        Driver = 3,
        Admin = 4
    }


    public enum CampaignStatus
    {
        Draft = 1,
        Active = 2,
        Completed = 3,
        Expired = 4,
        Cancelled = 5
    }


    public enum OrderStatus
    {
        PendingPayment = 1,
        Paid = 2,
        Confirmed = 3,
        Preparing = 4,
        AssignedToDriver = 5,
        OutForDelivery = 6,
        Delivered = 7,
        Cancelled = 8,
        Refunded = 9
    }
```

```
    public enum PaymentStatus
    {
        Pending = 1,
        Paid = 2,
        Failed = 3,
        Refunded = 4,
        PartiallyRefunded = 5
    }


    public enum MerchantOrderStatus
    {
        Pending = 1,
        Preparing = 2,
        ReadyForPickup = 3,
        HandedToDriver = 4,
        Completed = 5,
        Cancelled = 6
    }


    public enum DeliveryStatus
    {
        Pending = 1,
        Assigned = 2,
        PickedUp = 3,
        OutForDelivery = 4,
        Delivered = 5,
        Failed = 6,
        Cancelled = 7
    }


    //
============================================================
    // DB CONTEXT
    //
============================================================

    public class ApplicationDbContext
        : IdentityDbContext<ApplicationUser,
IdentityRole<Guid>, Guid>
    {
```

```
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<CustomerProfile> CustomerProfiles =>
Set<CustomerProfile>();
        public DbSet<MerchantProfile> MerchantProfiles =>
Set<MerchantProfile>();
        public DbSet<DriverProfile> DriverProfiles =>
Set<DriverProfile>();

        public DbSet<Region> Regions => Set<Region>();
        public DbSet<City> Cities => Set<City>();
        public DbSet<District> Districts => Set<District>();
        public DbSet<Address> Addresses => Set<Address>();

        public DbSet<Product> Products => Set<Product>();
        public DbSet<ProductImage> ProductImages =>
Set<ProductImage>();

        public DbSet<Campaign> Campaigns => Set<Campaign>();
        public DbSet<CampaignOrder> CampaignOrders =>
Set<CampaignOrder>();

        public DbSet<Payment> Payments => Set<Payment>();

        public DbSet<MerchantOrder> MerchantOrders =>
Set<MerchantOrder>();
        public DbSet<Delivery> Deliveries => Set<Delivery>();


        protected override void OnModelCreating(ModelBuilder
builder)
        {
            base.OnModelCreating(builder);

            //
==================================================
            // USER -> CUSTOMER PROFILE
            //
==================================================

            builder.Entity<CustomerProfile>()
```

```
                .HasOne(x => x.User)
                .WithOne(x => x.CustomerProfile)
                .HasForeignKey<CustomerProfile>(x =>
x.UserId)
                .OnDelete(DeleteBehavior.Cascade);


            //
=====================================================
            // USER -> MERCHANT PROFILE
            //
=====================================================

            builder.Entity<MerchantProfile>()
                .HasOne(x => x.User)
                .WithOne(x => x.MerchantProfile)
                .HasForeignKey<MerchantProfile>(x =>
x.UserId)
                .OnDelete(DeleteBehavior.Cascade);


            //
=====================================================
            // USER -> DRIVER PROFILE
            //
=====================================================

            builder.Entity<DriverProfile>()
                .HasOne(x => x.User)
                .WithOne(x => x.DriverProfile)
                .HasForeignKey<DriverProfile>(x => x.UserId)
                .OnDelete(DeleteBehavior.Cascade);


            //
=====================================================
            // REGION -> CITY
            //
=====================================================

            builder.Entity<Region>()
                .HasMany(x => x.Cities)
                .WithOne(x => x.Region)
                .HasForeignKey(x => x.RegionId)
                .OnDelete(DeleteBehavior.Restrict);
```

```
            //
============================================================
            // CITY -> DISTRICT
            //
============================================================

            builder.Entity<City>()
                .HasMany(x => x.Districts)
                .WithOne(x => x.City)
                .HasForeignKey(x => x.CityId)
                .OnDelete(DeleteBehavior.Restrict);


            //
============================================================
            // USER -> ADDRESSES
            //
============================================================

            builder.Entity<ApplicationUser>()
                .HasMany(x => x.Addresses)
                .WithOne(x => x.User)
                .HasForeignKey(x => x.UserId)
                .OnDelete(DeleteBehavior.Cascade);


            //
============================================================
            // REGION -> ADDRESS
            //
============================================================

            builder.Entity<Address>()
                .HasOne(x => x.Region)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.RegionId)
                .OnDelete(DeleteBehavior.Restrict);


            //
============================================================
            // CITY -> ADDRESS
```

```
            //
=====================================================

            builder.Entity<Address>()
                .HasOne(x => x.City)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.CityId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // DISTRICT -> ADDRESS
            //
=====================================================

            builder.Entity<Address>()
                .HasOne(x => x.District)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.DistrictId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // MERCHANT -> PRODUCTS
            //
=====================================================

            builder.Entity<Product>()
                .HasOne(x => x.Merchant)
                .WithMany()
                .HasForeignKey(x => x.MerchantId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // PRODUCT -> IMAGES
            //
=====================================================

            builder.Entity<Product>()
                .HasMany(x => x.Images)
                .WithOne(x => x.Product)
```

```
                .HasForeignKey(x => x.ProductId)
                .OnDelete(DeleteBehavior.Cascade);


            //
=========================================================
            // MERCHANT -> CAMPAIGNS
            //
=========================================================

            builder.Entity<Campaign>()
                .HasOne(x => x.Merchant)
                .WithMany(x => x.Campaigns)
                .HasForeignKey(x => x.MerchantId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=========================================================
            // PRODUCT -> CAMPAIGNS
            //
=========================================================

            builder.Entity<Campaign>()
                .HasOne(x => x.Product)
                .WithMany(x => x.Campaigns)
                .HasForeignKey(x => x.ProductId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=========================================================
            // CAMPAIGN -> CUSTOMER ORDERS
            //
=========================================================

            builder.Entity<CampaignOrder>()
                .HasOne(x => x.Campaign)
                .WithMany(x => x.Orders)
                .HasForeignKey(x => x.CampaignId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=========================================================
```

```
            // CUSTOMER -> CAMPAIGN ORDERS
            //
=====================================================

            builder.Entity<CampaignOrder>()
                .HasOne(x => x.Customer)
                .WithMany(x => x.CampaignOrders)
                .HasForeignKey(x => x.CustomerId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // ADDRESS -> CAMPAIGN ORDERS
            //
=====================================================

            builder.Entity<CampaignOrder>()
                .HasOne(x => x.Address)
                .WithMany()
                .HasForeignKey(x => x.AddressId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // CAMPAIGN ORDER -> PAYMENT
            //
=====================================================

            builder.Entity<Payment>()
                .HasOne(x => x.CampaignOrder)
                .WithOne(x => x.Payment)
                .HasForeignKey<Payment>(x =>
x.CampaignOrderId)
                .OnDelete(DeleteBehavior.Cascade);


            //
=====================================================
            // CAMPAIGN -> MERCHANT ORDER
            //
=====================================================

            builder.Entity<MerchantOrder>()
```

```
                .HasOne(x => x.Campaign)
                .WithOne(x => x.MerchantOrder)
                .HasForeignKey<MerchantOrder>(x =>
x.CampaignId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // MERCHANT -> MERCHANT ORDER
            //
=====================================================

            builder.Entity<MerchantOrder>()
                .HasOne(x => x.Merchant)
                .WithMany()
                .HasForeignKey(x => x.MerchantId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // MERCHANT ORDER -> DELIVERIES
            //
=====================================================

            builder.Entity<Delivery>()
                .HasOne(x => x.MerchantOrder)
                .WithMany(x => x.Deliveries)
                .HasForeignKey(x => x.MerchantOrderId)
                .OnDelete(DeleteBehavior.Restrict);


            //
=====================================================
            // CAMPAIGN ORDER -> DELIVERY
            //
=====================================================

            builder.Entity<Delivery>()
                .HasOne(x => x.CampaignOrder)
                .WithOne(x => x.Delivery)
                .HasForeignKey<Delivery>(x =>
x.CampaignOrderId)
                .OnDelete(DeleteBehavior.Restrict);
```

```
            //
============================================================
            // DRIVER -> DELIVERIES
            //
============================================================

            builder.Entity<Delivery>()
                .HasOne(x => x.Driver)
                .WithMany(x => x.Deliveries)
                .HasForeignKey(x => x.DriverId)
                .OnDelete(DeleteBehavior.Restrict);


            //
============================================================
            // DECIMAL PRECISION
            //
============================================================

            builder.Entity<Product>()
                .Property(x => x.Id)
                .ValueGeneratedOnAdd();

            builder.Entity<Campaign>()
                .Property(x => x.OriginalPrice)
                .HasPrecision(18, 2);

            builder.Entity<Campaign>()
                .Property(x => x.CampaignPrice)
                .HasPrecision(18, 2);

            builder.Entity<CampaignOrder>()
                .Property(x => x.UnitPrice)
                .HasPrecision(18, 2);

            builder.Entity<CampaignOrder>()
                .Property(x => x.TotalAmount)
                .HasPrecision(18, 2);

            builder.Entity<Payment>()
                .Property(x => x.Amount)
                .HasPrecision(18, 2);
```

```
            builder.Entity<MerchantOrder>()
                .Property(x => x.TotalAmount)
                .HasPrecision(18, 2);


            //
============================================================
            // INDEXES
            //
============================================================

            builder.Entity<Campaign>()
                .HasIndex(x => new
                {
                    x.Status,
                    x.StartDate,
                    x.EndDate
                });

            builder.Entity<CampaignOrder>()
                .HasIndex(x => new
                {
                    x.CustomerId,
                    x.CampaignId
                });

            builder.Entity<Address>()
                .HasIndex(x => x.UserId);

            builder.Entity<Product>()
                .HasIndex(x => x.MerchantId);

            builder.Entity<Delivery>()
                .HasIndex(x => new
                {
                    x.DriverId,
                    x.Status
                });


            //
============================================================
            // IDENTITY
            //
============================================================
```

```
            builder.Entity<ApplicationUser>()
                .HasIndex(x => x.PhoneNumber)
                .IsUnique();

            builder.Entity<MerchantProfile>()
                .HasIndex(x => x.UserId)
                .IsUnique();

            builder.Entity<CustomerProfile>()
                .HasIndex(x => x.UserId)
                .IsUnique();

            builder.Entity<DriverProfile>()
                .HasIndex(x => x.UserId)
                .IsUnique();

            builder.Entity<Payment>()
                .HasIndex(x => x.TransactionId);
        }
    }
}
```

ملاحظة مهمة في التصميم

أنا فصلت بين:

CampaignOrder
وهو طلب **العميل الفردي** داخل الحملة.

وبين:

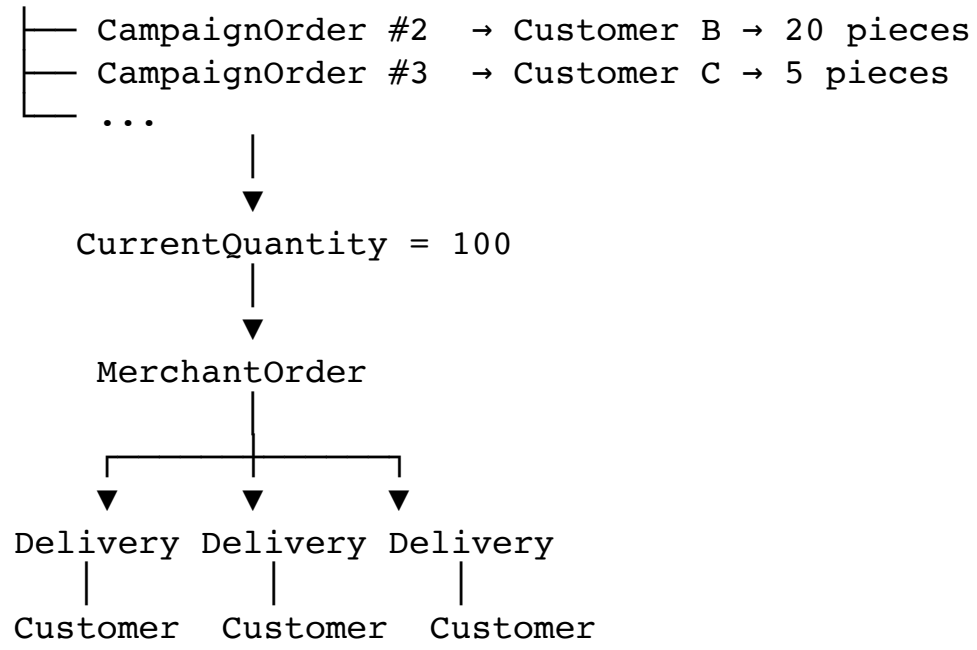
MerchantOrder
وهو **الطلب المجمع للتاجر** بعد اكتمال الحملة.

وبين:

Delivery
وهو عملية توصيل طلب العميل.

وبالتالي مثال حملة 100 قطعة:

```
Campaign
    │
    ├── CampaignOrder #1  → Customer A → 10 pieces
```



وهذا أفضل من جعل الطلب المجمع هو نفسه طلب العميل، لأن التاجر يحتاج أن يستلم **طلبًا واحدًا بإجمالي الكمية** بينما المنصة تحتاج الاحتفاظ بكل طلب عميل وعنوانه ودفعه بشكل مستقل.

كذلك استخدمت **Identity بـ `Guid`** بدل `string`، بحيث تكون كل الـ IDs متناسقة مع باقي النظام، ويمكنك استخدام:

```
[Authorize(Roles = "Merchant")]
```

للتاجر، و:

```
[Authorize(Roles = "Customer")]
```

للعميل، و:

```
[Authorize(Roles = "Driver")]
```

للمندوب.

بالنسبة لل Models c# انا اريد اقسام للمتاجر وكذلك product category وكذلك ال cart

نعم، والأفضل نضيفها للـ **C# Models** السابقة بطريقة تكون مناسبة لـ **MIGA** وتبقى قابلة للتوسع.

أقترح الهيكل التالي:

1. Store / Merchant Store

المتجر يكون تابعًا للتاجر، والتاجر يمكن أن يمتلك أكثر من متجر.

2. StoreCategory

أقسام المتاجر، مثل:

- مطاعم
- إلكترونيات
- أزياء
- بقالة
- عناية وجمال

3. ProductCategory

تصنيف المنتجات داخل المتجر، مثل:

- جوالات
- لابتوبات
- أحذية
- عطور

ويُفضل أن يدعم **ParentCategory** حتى نستطيع عمل تصنيفات فرعية.

4. Cart

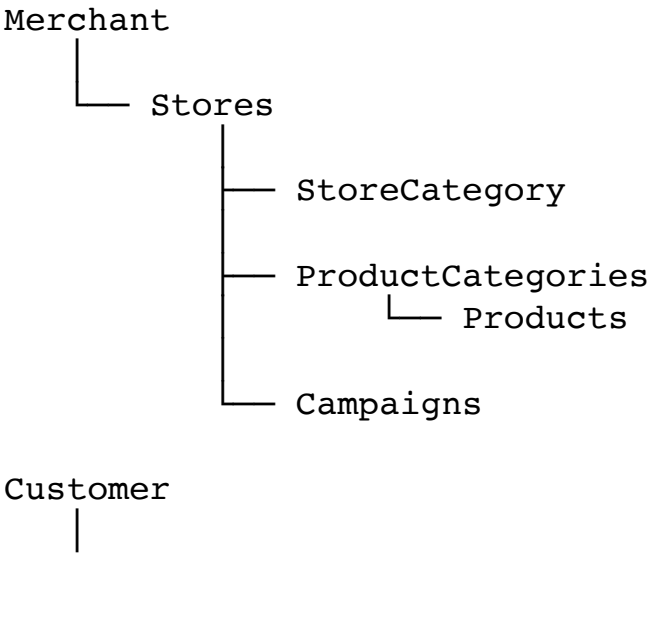
سلة العميل.

وبما أن مشروعك يعتمد على **Campaigns / الشراء الجماعي**، أنصح ألا نجعل الـ Cart مرتبطًا فقط بـ Product، بل يدعم:

- Product
- Campaign

بحيث العميل يستطيع إضافة منتج عادي أو الانضمام إلى حملة شراء جماعي.

العلاقات



```
    └── Cart
          └── CartItems
                ├── Product
                └── Campaign
```

Models المقترحة

```
public class Store
{
    public Guid Id { get; set; }

    public Guid MerchantId { get; set; }
    public MerchantProfile Merchant { get; set; } = null!;

    public Guid StoreCategoryId { get; set; }
    public StoreCategory StoreCategory { get; set; } = null!;

    public string Name { get; set; } = null!;
    public string? Description { get; set; }

    public string? LogoUrl { get; set; }
    public string? CoverImageUrl { get; set; }

    public bool IsActive { get; set; } = true;

    public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

    public ICollection<Product> Products { get; set; } = new
List<Product>();
    public ICollection<Campaign> Campaigns { get; set; } =
new List<Campaign>();
}


public class StoreCategory
{
    public Guid Id { get; set; }

    public string NameAr { get; set; } = null!;
    public string NameEn { get; set; } = null!;

    public string? IconUrl { get; set; }
```

```
    public bool IsActive { get; set; } = true;

    public ICollection<Store> Stores { get; set; } = new
List<Store>();
}


public class ProductCategory
{
    public Guid Id { get; set; }

    public Guid StoreId { get; set; }
    public Store Store { get; set; } = null!;

    public Guid? ParentCategoryId { get; set; }
    public ProductCategory? ParentCategory { get; set; }

    public string NameAr { get; set; } = null!;
    public string NameEn { get; set; } = null!;

    public string? ImageUrl { get; set; }

    public int SortOrder { get; set; }

    public bool IsActive { get; set; } = true;

    public ICollection<ProductCategory> Children { get;
set; } = new List<ProductCategory>();

    public ICollection<Product> Products { get; set; } = new
List<Product>();
}
```

ونعدل Product الموجودة لتصبح:

```
public class Product
{
    public Guid Id { get; set; }

    public Guid StoreId { get; set; }
    public Store Store { get; set; } = null!;

    public Guid ProductCategoryId { get; set; }
```

```
    public ProductCategory ProductCategory { get; set; } =
null!;

    public string NameAr { get; set; } = null!;
    public string NameEn { get; set; } = null!;

    public string? DescriptionAr { get; set; }
    public string? DescriptionEn { get; set; }

    public decimal Price { get; set; }

    public int StockQuantity { get; set; }

    public bool IsActive { get; set; } = true;

    public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

    public ICollection<ProductImage> Images { get; set; } =
new List<ProductImage>();

    public ICollection<Campaign> Campaigns { get; set; } =
new List<Campaign>();
}
```

Cart

```
public class Cart
{
    public Guid Id { get; set; }

    public Guid CustomerId { get; set; }
    public CustomerProfile Customer { get; set; } = null!;

    public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    public DateTime UpdatedAt { get; set; } =
DateTime.UtcNow;

    public ICollection<CartItem> Items { get; set; } = new
List<CartItem>();
}
```

```
public class CartItem
{
    public Guid Id { get; set; }

    public Guid CartId { get; set; }
    public Cart Cart { get; set; } = null!;

    public Guid ProductId { get; set; }
    public Product Product { get; set; } = null!;

    // Optional: إذا كان المنتج مضافًا من حملة
    public Guid? CampaignId { get; set; }
    public Campaign? Campaign { get; set; }

    public int Quantity { get; set; }

    public decimal UnitPrice { get; set; }

    public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
}
```

نقطة مهمة جدًا في MIGA

أنا أنصح أن يكون عندك **Store** مستقل عن `MerchantProfile`.

يعني:

```
Merchant
   ↓
Store
   ↓
Products
   ↓
Campaigns
```

لأن مستقبلاً ممكن تاجر واحد يكون عنده:

```
Merchant
  ├── MIGA Fashion
  ├── MIGA Electronics
  └── MIGA Home
```

وهذا أفضل بكثير من ربط المنتجات مباشرة بالتاجر.

وبالنسبة لـ **Cart**، هناك قرار معماري مهم: هل تسمح للعميل أن يضع منتجات من **عدة متاجر في سلة واحدة** أم كل متجر له Cart مستقل؟ لمشروع مثل MIGA، أميل إلى **Cart واحدة لكن يتم تقسيمها تلقائيًا حسب Store عند Checkout**، خصوصًا لأن الطلبات والدفع والتوصيل قد تختلف من متجر لآخر.

اريدك تكتب ال models كلها

أكيد. سأجمع لك **كل الـ Models الخاصة بـ MIGA في ملف C# واحد**، بحيث تكون الصورة كاملة ومترابطة، وتشمل:

- ASP.NET Core Identity
- Customer / Merchant / Driver
- Store / Store Categories
- Product / Product Categories / Product Images
- Campaign / Group Buying
- Cart / Cart Items
- Campaign Orders
- Payments
- Merchant Orders
- Deliveries
- Region → City → District → Address
- Enums
- `ApplicationDbContext`
- العلاقات والـ indexes والـ decimal precision

وأنا سأبنيها بحيث يكون **Merchant لديه أكثر من Store**، وكل Store لديه منتجات وحملات وأقسام.

```
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace Miga.Domain
{
    //
================================================================
    // ENUMS
    //
================================================================

    public enum UserType
    {
        Customer = 1,
        Merchant = 2,
```

```
    Driver = 3,
    Admin = 4
}

public enum StoreStatus
{
    Active = 1,
    Inactive = 2,
    Suspended = 3
}

public enum CampaignStatus
{
    Draft = 1,
    Active = 2,
    Completed = 3,
    Failed = 4,
    Cancelled = 5,
    Processing = 6,
    Closed = 7
}

public enum CampaignOrderStatus
{
    Pending = 1,
    Paid = 2,
    Confirmed = 3,
    Preparing = 4,
    ReadyForDelivery = 5,
    OutForDelivery = 6,
    Delivered = 7,
    Cancelled = 8,
    Refunded = 9
}

public enum PaymentStatus
{
    Pending = 1,
    Paid = 2,
    Failed = 3,
    Cancelled = 4,
    Refunded = 5,
    PartiallyRefunded = 6
}
```

```
    public enum MerchantOrderStatus
    {
        Pending = 1,
        SentToMerchant = 2,
        Confirmed = 3,
        Preparing = 4,
        Ready = 5,
        Completed = 6,
        Cancelled = 7
    }

    public enum DeliveryStatus
    {
        Pending = 1,
        Assigned = 2,
        DriverAccepted = 3,
        PickedUp = 4,
        OutForDelivery = 5,
        Delivered = 6,
        Failed = 7,
        Cancelled = 8
    }

    public enum OrderType
    {
        Product = 1,
        Campaign = 2
    }


    //
============================================================
    // APPLICATION USER - IDENTITY
    //
============================================================

    public class ApplicationUser : IdentityUser<Guid>
    {
        public string FullName { get; set; } = null!;

        public UserType UserType { get; set; }

        public bool IsActive { get; set; } = true;
```

```
        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        // Profiles
        public CustomerProfile? CustomerProfile { get; set; }

        public MerchantProfile? MerchantProfile { get; set; }

        public DriverProfile? DriverProfile { get; set; }

        // Addresses
        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();

        // Merchant campaigns
        public ICollection<Campaign> Campaigns { get; set; }
            = new List<Campaign>();

        // Campaign orders
        public ICollection<CampaignOrder> CampaignOrders
{ get; set; }
            = new List<CampaignOrder>();

        // Deliveries
        public ICollection<Delivery> Deliveries { get; set; }
            = new List<Delivery>();
    }


    //
============================================================
    // CUSTOMER
    //
============================================================

    public class CustomerProfile
    {
        public Guid Id { get; set; }

        public Guid UserId { get; set; }

        public ApplicationUser User { get; set; } = null!;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
```

```
        public ICollection<Cart> Carts { get; set; }
            = new List<Cart>();

        public ICollection<CampaignOrder> CampaignOrders
{ get; set; }
            = new List<CampaignOrder>();
    }


    //
============================================================
    // MERCHANT
    //
============================================================

    public class MerchantProfile
    {
        public Guid Id { get; set; }

        public Guid UserId { get; set; }

        public ApplicationUser User { get; set; } = null!;

        public string? BusinessNameAr { get; set; }

        public string? BusinessNameEn { get; set; }

        public string? CommercialRegistrationNumber { get;
set; }

        public string? TaxNumber { get; set; }

        public bool IsVerified { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<Store> Stores { get; set; }
            = new List<Store>();
    }


    //
============================================================
```

```
    // DRIVER
    //
============================================================

    public class DriverProfile
    {
        public Guid Id { get; set; }

        public Guid UserId { get; set; }

        public ApplicationUser User { get; set; } = null!;

        public string? VehicleType { get; set; }

        public string? VehiclePlateNumber { get; set; }

        public bool IsAvailable { get; set; }

        public bool IsVerified { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
    // LOCATION
    // Region → City → District
    //
============================================================

    public class Region
    {
        public Guid Id { get; set; }

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public ICollection<City> Cities { get; set; }
            = new List<City>();

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();
```

```
    }


    public class City
    {
        public Guid Id { get; set; }

        public Guid RegionId { get; set; }

        public Region Region { get; set; } = null!;

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public ICollection<District> Districts { get; set; }
            = new List<District>();

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();
    }


    public class District
    {
        public Guid Id { get; set; }

        public Guid CityId { get; set; }

        public City City { get; set; } = null!;

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public ICollection<Address> Addresses { get; set; }
            = new List<Address>();
    }


    //
=============================================================
    // ADDRESS
    //
=============================================================
```

```
    public class Address
    {
        public Guid Id { get; set; }

        public Guid UserId { get; set; }

        public ApplicationUser User { get; set; } = null!;

        public Guid RegionId { get; set; }

        public Region Region { get; set; } = null!;

        public Guid CityId { get; set; }

        public City City { get; set; } = null!;

        public Guid DistrictId { get; set; }

        public District District { get; set; } = null!;

        public string AddressName { get; set; } = null!;

        public string AddressDetails { get; set; } = null!;

        public string? BuildingNumber { get; set; }

        public string? StreetName { get; set; }

        public string? PostalCode { get; set; }

        public string? AdditionalDirections { get; set; }

        public decimal Latitude { get; set; }

        public decimal Longitude { get; set; }

        public bool IsDefault { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }
```

```
    //
============================================================
    // STORE CATEGORY
    // مثال:
    // Restaurants
    // Electronics
    // Fashion
    // Grocery
    // Beauty
    //
============================================================

    public class StoreCategory
    {
        public Guid Id { get; set; }

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public string? DescriptionAr { get; set; }

        public string? DescriptionEn { get; set; }

        public string? IconUrl { get; set; }

        public string? ImageUrl { get; set; }

        public int SortOrder { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<Store> Stores { get; set; }
            = new List<Store>();
    }


    //
============================================================
    // STORE
    //
============================================================
```

```
    public class Store
    {
        public Guid Id { get; set; }

        public Guid MerchantId { get; set; }

        public MerchantProfile Merchant { get; set; } =
null!;

        public Guid StoreCategoryId { get; set; }

        public StoreCategory StoreCategory { get; set; } =
null!;

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public string? DescriptionAr { get; set; }

        public string? DescriptionEn { get; set; }

        public string? LogoUrl { get; set; }

        public string? CoverImageUrl { get; set; }

        public StoreStatus Status { get; set; } =
StoreStatus.Active;

        public bool IsVerified { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<ProductCategory> ProductCategories
{ get; set; }
            = new List<ProductCategory>();

        public ICollection<Product> Products { get; set; }
            = new List<Product>();

        public ICollection<Campaign> Campaigns { get; set; }
            = new List<Campaign>();
    }
```

```
    //
============================================================
    // PRODUCT CATEGORY
    // Supports Parent / Child categories
    //
============================================================

    public class ProductCategory
    {
        public Guid Id { get; set; }

        public Guid StoreId { get; set; }

        public Store Store { get; set; } = null!;

        public Guid? ParentCategoryId { get; set; }

        public ProductCategory? ParentCategory { get; set; }

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public string? DescriptionAr { get; set; }

        public string? DescriptionEn { get; set; }

        public string? ImageUrl { get; set; }

        public int SortOrder { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<ProductCategory> Children { get;
set; }
            = new List<ProductCategory>();

        public ICollection<Product> Products { get; set; }
            = new List<Product>();
    }
```

```
    //
============================================================
    // PRODUCT
    //
============================================================

    public class Product
    {
        public Guid Id { get; set; }

        public Guid StoreId { get; set; }

        public Store Store { get; set; } = null!;

        public Guid ProductCategoryId { get; set; }

        public ProductCategory ProductCategory { get; set; }
= null!;

        public string NameAr { get; set; } = null!;

        public string NameEn { get; set; } = null!;

        public string? DescriptionAr { get; set; }

        public string? DescriptionEn { get; set; }

        public decimal Price { get; set; }

        public int StockQuantity { get; set; }

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<ProductImage> Images { get; set; }
            = new List<ProductImage>();

        public ICollection<Campaign> Campaigns { get; set; }
            = new List<Campaign>();

        public ICollection<CartItem> CartItems { get; set; }
```

```
            = new List<CartItem>();
    }


    //
============================================================
    // PRODUCT IMAGE
    //
============================================================

    public class ProductImage
    {
        public Guid Id { get; set; }

        public Guid ProductId { get; set; }

        public Product Product { get; set; } = null!;

        public string ImageUrl { get; set; } = null!;

        public int SortOrder { get; set; }

        public bool IsMain { get; set; }
    }


    //
============================================================
    // CAMPAIGN
    // Group Buying Campaign
    //
============================================================

    public class Campaign
    {
        public Guid Id { get; set; }

        public Guid StoreId { get; set; }

        public Store Store { get; set; } = null!;

        public Guid ProductId { get; set; }

        public Product Product { get; set; } = null!;
```

```
        public Guid MerchantUserId { get; set; }

        public ApplicationUser MerchantUser { get; set; } =
null!;

        public string TitleAr { get; set; } = null!;

        public string TitleEn { get; set; } = null!;

        public string? DescriptionAr { get; set; }

        public string? DescriptionEn { get; set; }

        // Original product price
        public decimal OriginalPrice { get; set; }

        // Group buying price
        public decimal CampaignPrice { get; set; }

        // Minimum quantity required
        public int TargetQuantity { get; set; }

        // Current participants
        public int CurrentQuantity { get; set; }

        public DateTime StartDate { get; set; }

        public DateTime EndDate { get; set; }

        public CampaignStatus Status { get; set; } =
CampaignStatus.Draft;

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<CampaignOrder> Orders { get;
set; }
            = new List<CampaignOrder>();

        public MerchantOrder? MerchantOrder { get; set; }
    }
```

```
    //
============================================================
    // CART
    //
============================================================

    public class Cart
    {
        public Guid Id { get; set; }

        public Guid CustomerId { get; set; }

        public CustomerProfile Customer { get; set; } =
null!;

        public bool IsActive { get; set; } = true;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public DateTime UpdatedAt { get; set; } =
DateTime.UtcNow;

        public ICollection<CartItem> Items { get; set; }
            = new List<CartItem>();
    }


    //
============================================================
    // CART ITEM
    //
============================================================

    public class CartItem
    {
        public Guid Id { get; set; }

        public Guid CartId { get; set; }

        public Cart Cart { get; set; } = null!;

        public Guid ProductId { get; set; }

        public Product Product { get; set; } = null!;
```

```
        // Nullable because this can be a normal product
        // or a campaign product.
        public Guid? CampaignId { get; set; }

        public Campaign? Campaign { get; set; }

        public int Quantity { get; set; }

        // Price at the moment of adding to cart
        public decimal UnitPrice { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public DateTime UpdatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
    // CAMPAIGN ORDER
    // One customer participating in one campaign
    //
============================================================

    public class CampaignOrder
    {
        public Guid Id { get; set; }

        public Guid CampaignId { get; set; }

        public Campaign Campaign { get; set; } = null!;

        public Guid CustomerId { get; set; }

        public ApplicationUser Customer { get; set; } =
null!;

        public Guid? CustomerProfileId { get; set; }

        public CustomerProfile? CustomerProfile { get; set; }

        public Guid DeliveryAddressId { get; set; }
```

```
        public Address DeliveryAddress { get; set; } = null!;

        public int Quantity { get; set; }

        public decimal UnitPrice { get; set; }

        public decimal TotalAmount { get; set; }

        public CampaignOrderStatus Status { get; set; }
            = CampaignOrderStatus.Pending;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public DateTime? ConfirmedAt { get; set; }

        public DateTime? DeliveredAt { get; set; }

        public Payment? Payment { get; set; }

        public Delivery? Delivery { get; set; }
    }


    //
============================================================
    // PAYMENT
    //
============================================================

    public class Payment
    {
        public Guid Id { get; set; }

        public Guid CampaignOrderId { get; set; }

        public CampaignOrder CampaignOrder { get; set; } =
null!;

        public decimal Amount { get; set; }

        public string? PaymentMethod { get; set; }

        public string? TransactionId { get; set; }
```

```
        public PaymentStatus Status { get; set; }
            = PaymentStatus.Pending;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public DateTime? PaidAt { get; set; }

        public DateTime? RefundedAt { get; set; }
    }


    //
============================================================
    // MERCHANT ORDER
    //
    // Created when campaign reaches target quantity.
    //
    // Example:
    //
    // Campaign target = 100
    //
    // Customer 1 = 2
    // Customer 2 = 5
    // Customer 3 = 1
    //
    // MerchantOrder = aggregated order for merchant
    //
============================================================

    public class MerchantOrder
    {
        public Guid Id { get; set; }

        public Guid CampaignId { get; set; }

        public Campaign Campaign { get; set; } = null!;

        public Guid StoreId { get; set; }

        public Store Store { get; set; } = null!;

        public int TotalQuantity { get; set; }
```

```
        public decimal TotalAmount { get; set; }

        public MerchantOrderStatus Status { get; set; }
            = MerchantOrderStatus.Pending;

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;

        public DateTime? ConfirmedAt { get; set; }

        public DateTime? ReadyAt { get; set; }

        public DateTime? CompletedAt { get; set; }

        public ICollection<Delivery> Deliveries { get; set; }
            = new List<Delivery>();
    }


    //
============================================================
    // DELIVERY
    //
============================================================

    public class Delivery
    {
        public Guid Id { get; set; }

        public Guid CampaignOrderId { get; set; }

        public CampaignOrder CampaignOrder { get; set; } =
null!;

        public Guid MerchantOrderId { get; set; }

        public MerchantOrder MerchantOrder { get; set; } =
null!;

        public Guid? DriverUserId { get; set; }

        public ApplicationUser? DriverUser { get; set; }

        public Guid DeliveryAddressId { get; set; }
```

```
        public Address DeliveryAddress { get; set; } = null!;

        public DeliveryStatus Status { get; set; }
            = DeliveryStatus.Pending;

        public DateTime? AssignedAt { get; set; }

        public DateTime? PickedUpAt { get; set; }

        public DateTime? OutForDeliveryAt { get; set; }

        public DateTime? DeliveredAt { get; set; }

        public string? DeliveryNotes { get; set; }

        public DateTime CreatedAt { get; set; } =
DateTime.UtcNow;
    }


    //
============================================================
    // DATABASE CONTEXT
    //
============================================================

    public class ApplicationDbContext
        : IdentityDbContext<
            ApplicationUser,
            IdentityRole<Guid>,
            Guid>
    {
        public ApplicationDbContext(
            DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        // Users / Profiles
        public DbSet<CustomerProfile> CustomerProfiles =>
Set<CustomerProfile>();
        public DbSet<MerchantProfile> MerchantProfiles =>
Set<MerchantProfile>();
        public DbSet<DriverProfile> DriverProfiles =>
Set<DriverProfile>();
```

```
        // Location
        public DbSet<Region> Regions => Set<Region>();
        public DbSet<City> Cities => Set<City>();
        public DbSet<District> Districts => Set<District>();
        public DbSet<Address> Addresses => Set<Address>();

        // Stores
        public DbSet<Store> Stores => Set<Store>();
        public DbSet<StoreCategory> StoreCategories =>
Set<StoreCategory>();

        // Products
        public DbSet<Product> Products => Set<Product>();
        public DbSet<ProductCategory> ProductCategories =>
Set<ProductCategory>();
        public DbSet<ProductImage> ProductImages =>
Set<ProductImage>();

        // Campaigns
        public DbSet<Campaign> Campaigns => Set<Campaign>();
        public DbSet<CampaignOrder> CampaignOrders =>
Set<CampaignOrder>();

        // Cart
        public DbSet<Cart> Carts => Set<Cart>();
        public DbSet<CartItem> CartItems => Set<CartItem>();

        // Orders / Payments / Delivery
        public DbSet<Payment> Payments => Set<Payment>();
        public DbSet<MerchantOrder> MerchantOrders =>
Set<MerchantOrder>();
        public DbSet<Delivery> Deliveries => Set<Delivery>();


        protected override void OnModelCreating(ModelBuilder
builder)
        {
            base.OnModelCreating(builder);


            //
==================================================
            // APPLICATION USER
```

```
            //
============================================================

            builder.Entity<ApplicationUser>(entity =>
            {
                entity.Property(x => x.FullName)
                    .HasMaxLength(200)
                    .IsRequired();

                entity.Property(x => x.UserType)
                    .IsRequired();

                entity.HasIndex(x => x.PhoneNumber)
                    .IsUnique()
                    .HasFilter("[PhoneNumber] IS NOT NULL");
            });


            //
============================================================
            // CUSTOMER PROFILE
            //
============================================================

            builder.Entity<CustomerProfile>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.HasIndex(x => x.UserId)
                    .IsUnique();

                entity.HasOne(x => x.User)
                    .WithOne(x => x.CustomerProfile)
                    .HasForeignKey<CustomerProfile>(x =>
x.UserId)
                    .OnDelete(DeleteBehavior.Cascade);
            });


            //
============================================================
            // MERCHANT PROFILE
            //
============================================================
```

```
            builder.Entity<MerchantProfile>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.HasIndex(x => x.UserId)
                    .IsUnique();

                entity.Property(x => x.BusinessNameAr)
                    .HasMaxLength(200);

                entity.Property(x => x.BusinessNameEn)
                    .HasMaxLength(200);

                entity.Property(x =>
x.CommercialRegistrationNumber)
                    .HasMaxLength(100);

                entity.Property(x => x.TaxNumber)
                    .HasMaxLength(100);

                entity.HasOne(x => x.User)
                    .WithOne(x => x.MerchantProfile)
                    .HasForeignKey<MerchantProfile>(x =>
x.UserId)
                    .OnDelete(DeleteBehavior.Cascade);
            });


            //
==================================================
            // DRIVER PROFILE
            //
==================================================

            builder.Entity<DriverProfile>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.HasIndex(x => x.UserId)
                    .IsUnique();

                entity.Property(x => x.VehicleType)
                    .HasMaxLength(100);

                entity.Property(x => x.VehiclePlateNumber)
```

```
                .HasMaxLength(50);

            entity.HasOne(x => x.User)
                .WithOne(x => x.DriverProfile)
                .HasForeignKey<DriverProfile>(x =>
x.UserId)
                .OnDelete(DeleteBehavior.Cascade);
        });


        //
=====================================================
        // REGION
        //
=====================================================

        builder.Entity<Region>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.Property(x => x.NameAr)
                .HasMaxLength(150)
                .IsRequired();

            entity.Property(x => x.NameEn)
                .HasMaxLength(150)
                .IsRequired();

            entity.HasIndex(x => x.NameAr);
        });


        //
=====================================================
        // CITY
        //
=====================================================

        builder.Entity<City>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.Property(x => x.NameAr)
                .HasMaxLength(150)
                .IsRequired();
```

```
                entity.Property(x => x.NameEn)
                    .HasMaxLength(150)
                    .IsRequired();

                entity.HasOne(x => x.Region)
                    .WithMany(x => x.Cities)
                    .HasForeignKey(x => x.RegionId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasIndex(x => new
                {
                    x.RegionId,
                    x.NameAr
                })
                .IsUnique();
            });


            //
=========================================================
            // DISTRICT
            //
=========================================================

            builder.Entity<District>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.NameAr)
                    .HasMaxLength(150)
                    .IsRequired();

                entity.Property(x => x.NameEn)
                    .HasMaxLength(150)
                    .IsRequired();

                entity.HasOne(x => x.City)
                    .WithMany(x => x.Districts)
                    .HasForeignKey(x => x.CityId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasIndex(x => new
                {
                    x.CityId,
```

```
                x.NameAr
            })
            .IsUnique();
        });


        //
=========================================================
        // ADDRESS
        //
=========================================================

        builder.Entity<Address>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.Property(x => x.AddressName)
                .HasMaxLength(100)
                .IsRequired();

            entity.Property(x => x.AddressDetails)
                .HasMaxLength(500)
                .IsRequired();

            entity.Property(x => x.BuildingNumber)
                .HasMaxLength(50);

            entity.Property(x => x.StreetName)
                .HasMaxLength(200);

            entity.Property(x => x.PostalCode)
                .HasMaxLength(20);

            entity.Property(x => x.AdditionalDirections)
                .HasMaxLength(500);

            entity.Property(x => x.Latitude)
                .HasPrecision(10, 7);

            entity.Property(x => x.Longitude)
                .HasPrecision(10, 7);

            entity.HasOne(x => x.User)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.UserId)
```

```
                .OnDelete(DeleteBehavior.Cascade);

            entity.HasOne(x => x.Region)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.RegionId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(x => x.City)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.CityId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(x => x.District)
                .WithMany(x => x.Addresses)
                .HasForeignKey(x => x.DistrictId)
                .OnDelete(DeleteBehavior.Restrict);
        });


        //
=====================================================
        // STORE CATEGORY
        //
=====================================================

        builder.Entity<StoreCategory>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.Property(x => x.NameAr)
                .HasMaxLength(150)
                .IsRequired();

            entity.Property(x => x.NameEn)
                .HasMaxLength(150)
                .IsRequired();

            entity.Property(x => x.DescriptionAr)
                .HasMaxLength(500);

            entity.Property(x => x.DescriptionEn)
                .HasMaxLength(500);

            entity.HasIndex(x => x.NameEn)
                .IsUnique();
```

```
        });


        //
==========================================================
        // STORE
        //
==========================================================

        builder.Entity<Store>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.Property(x => x.NameAr)
                .HasMaxLength(200)
                .IsRequired();

            entity.Property(x => x.NameEn)
                .HasMaxLength(200)
                .IsRequired();

            entity.Property(x => x.DescriptionAr)
                .HasMaxLength(1000);

            entity.Property(x => x.DescriptionEn)
                .HasMaxLength(1000);

            entity.HasOne(x => x.Merchant)
                .WithMany(x => x.Stores)
                .HasForeignKey(x => x.MerchantId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(x => x.StoreCategory)
                .WithMany(x => x.Stores)
                .HasForeignKey(x => x.StoreCategoryId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasIndex(x => new
            {
                x.MerchantId,
                x.NameEn
            });
        });
```

```
            //
============================================================
            // PRODUCT CATEGORY
            //
============================================================

            builder.Entity<ProductCategory>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.NameAr)
                    .HasMaxLength(150)
                    .IsRequired();

                entity.Property(x => x.NameEn)
                    .HasMaxLength(150)
                    .IsRequired();

                entity.Property(x => x.DescriptionAr)
                    .HasMaxLength(500);

                entity.Property(x => x.DescriptionEn)
                    .HasMaxLength(500);

                entity.HasOne(x => x.Store)
                    .WithMany(x => x.ProductCategories)
                    .HasForeignKey(x => x.StoreId)
                    .OnDelete(DeleteBehavior.Cascade);

                entity.HasOne(x => x.ParentCategory)
                    .WithMany(x => x.Children)
                    .HasForeignKey(x => x.ParentCategoryId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasIndex(x => new
                {
                    x.StoreId,
                    x.NameEn
                });
            });


            //
============================================================
            // PRODUCT
```

```
            //
==========================================================

            builder.Entity<Product>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.NameAr)
                    .HasMaxLength(250)
                    .IsRequired();

                entity.Property(x => x.NameEn)
                    .HasMaxLength(250)
                    .IsRequired();

                entity.Property(x => x.DescriptionAr)
                    .HasMaxLength(2000);

                entity.Property(x => x.DescriptionEn)
                    .HasMaxLength(2000);

                entity.Property(x => x.Price)
                    .HasPrecision(18, 2);

                entity.HasOne(x => x.Store)
                    .WithMany(x => x.Products)
                    .HasForeignKey(x => x.StoreId)
                    .OnDelete(DeleteBehavior.Cascade);

                entity.HasOne(x => x.ProductCategory)
                    .WithMany(x => x.Products)
                    .HasForeignKey(x => x.ProductCategoryId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasIndex(x => new
                {
                    x.StoreId,
                    x.ProductCategoryId
                });
            });


            //
==========================================================
            // PRODUCT IMAGE
```

```
            //
==========================================================

            builder.Entity<ProductImage>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.ImageUrl)
                    .HasMaxLength(1000)
                    .IsRequired();

                entity.HasOne(x => x.Product)
                    .WithMany(x => x.Images)
                    .HasForeignKey(x => x.ProductId)
                    .OnDelete(DeleteBehavior.Cascade);
            });


            //
==========================================================
            // CAMPAIGN
            //
==========================================================

            builder.Entity<Campaign>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.TitleAr)
                    .HasMaxLength(250)
                    .IsRequired();

                entity.Property(x => x.TitleEn)
                    .HasMaxLength(250)
                    .IsRequired();

                entity.Property(x => x.DescriptionAr)
                    .HasMaxLength(2000);

                entity.Property(x => x.DescriptionEn)
                    .HasMaxLength(2000);

                entity.Property(x => x.OriginalPrice)
                    .HasPrecision(18, 2);
```

```
            entity.Property(x => x.CampaignPrice)
                .HasPrecision(18, 2);

            entity.HasOne(x => x.Store)
                .WithMany(x => x.Campaigns)
                .HasForeignKey(x => x.StoreId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(x => x.Product)
                .WithMany(x => x.Campaigns)
                .HasForeignKey(x => x.ProductId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(x => x.MerchantUser)
                .WithMany(x => x.Campaigns)
                .HasForeignKey(x => x.MerchantUserId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasIndex(x => new
            {
                x.StoreId,
                x.Status
            });

            entity.HasIndex(x => new
            {
                x.StartDate,
                x.EndDate
            });
        });


        //
=====================================================
        // CART
        //
=====================================================

        builder.Entity<Cart>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.HasOne(x => x.Customer)
                .WithMany(x => x.Carts)
                .HasForeignKey(x => x.CustomerId)
```

```
                .OnDelete(DeleteBehavior.Cascade);

            entity.HasIndex(x => new
            {
                x.CustomerId,
                x.IsActive
            });
        });


        //
============================================================
        // CART ITEM
        //
============================================================

        builder.Entity<CartItem>(entity =>
        {
            entity.HasKey(x => x.Id);

            entity.Property(x => x.UnitPrice)
                .HasPrecision(18, 2);

            entity.HasOne(x => x.Cart)
                .WithMany(x => x.Items)
                .HasForeignKey(x => x.CartId)
                .OnDelete(DeleteBehavior.Cascade);

            entity.HasOne(x => x.Product)
                .WithMany(x => x.CartItems)
                .HasForeignKey(x => x.ProductId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasOne(x => x.Campaign)
                .WithMany()
                .HasForeignKey(x => x.CampaignId)
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasIndex(x => new
            {
                x.CartId,
                x.ProductId,
                x.CampaignId
            });
        });
```

```
            //
============================================================
            // CAMPAIGN ORDER
            //
============================================================

            builder.Entity<CampaignOrder>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.UnitPrice)
                    .HasPrecision(18, 2);

                entity.Property(x => x.TotalAmount)
                    .HasPrecision(18, 2);

                entity.HasOne(x => x.Campaign)
                    .WithMany(x => x.Orders)
                    .HasForeignKey(x => x.CampaignId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasOne(x => x.Customer)
                    .WithMany(x => x.CampaignOrders)
                    .HasForeignKey(x => x.CustomerId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasOne(x => x.CustomerProfile)
                    .WithMany(x => x.CampaignOrders)
                    .HasForeignKey(x => x.CustomerProfileId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasOne(x => x.DeliveryAddress)
                    .WithMany()
                    .HasForeignKey(x => x.DeliveryAddressId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasIndex(x => new
                {
                    x.CampaignId,
                    x.CustomerId
                });
            });
```

```
            //
============================================================
            // PAYMENT
            //
============================================================

            builder.Entity<Payment>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.Amount)
                    .HasPrecision(18, 2);

                entity.Property(x => x.PaymentMethod)
                    .HasMaxLength(100);

                entity.Property(x => x.TransactionId)
                    .HasMaxLength(200);

                entity.HasOne(x => x.CampaignOrder)
                    .WithOne(x => x.Payment)
                    .HasForeignKey<Payment>(x =>
x.CampaignOrderId)
                    .OnDelete(DeleteBehavior.Cascade);

                entity.HasIndex(x => x.TransactionId);
            });


            //
============================================================
            // MERCHANT ORDER
            //
============================================================

            builder.Entity<MerchantOrder>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.TotalAmount)
                    .HasPrecision(18, 2);

                entity.HasOne(x => x.Campaign)
                    .WithOne(x => x.MerchantOrder)
```

```
                    .HasForeignKey<MerchantOrder>(x =>
x.CampaignId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasOne(x => x.Store)
                    .WithMany()
                    .HasForeignKey(x => x.StoreId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasIndex(x => x.StoreId);
            });


            //
==========================================================
            // DELIVERY
            //
==========================================================

            builder.Entity<Delivery>(entity =>
            {
                entity.HasKey(x => x.Id);

                entity.Property(x => x.DeliveryNotes)
                    .HasMaxLength(1000);

                entity.HasOne(x => x.CampaignOrder)
                    .WithOne(x => x.Delivery)
                    .HasForeignKey<Delivery>(x =>
x.CampaignOrderId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasOne(x => x.MerchantOrder)
                    .WithMany(x => x.Deliveries)
                    .HasForeignKey(x => x.MerchantOrderId)
                    .OnDelete(DeleteBehavior.Restrict);

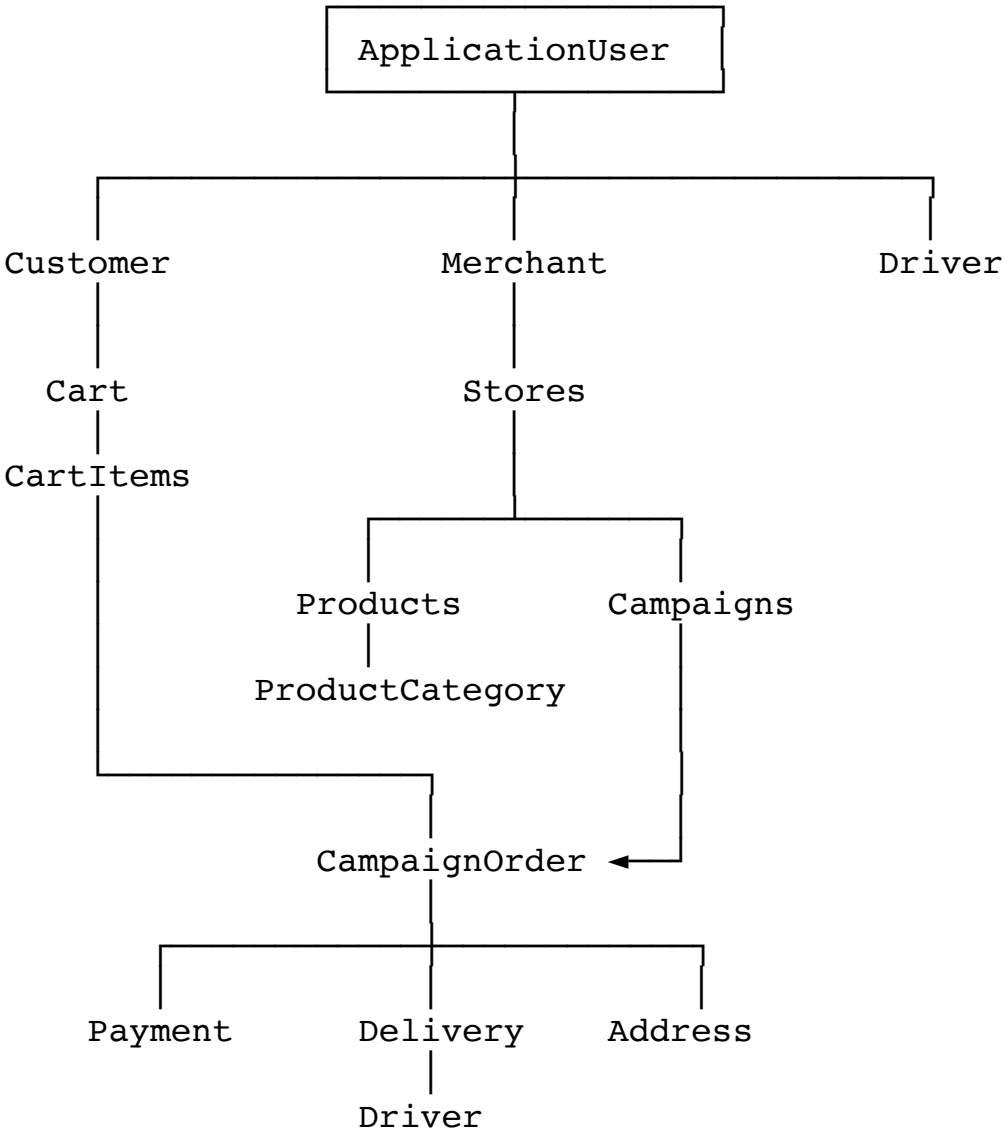
                entity.HasOne(x => x.DriverUser)
                    .WithMany(x => x.Deliveries)
                    .HasForeignKey(x => x.DriverUserId)
                    .OnDelete(DeleteBehavior.Restrict);

                entity.HasOne(x => x.DeliveryAddress)
                    .WithMany()
                    .HasForeignKey(x => x.DeliveryAddressId)
```

```
                .OnDelete(DeleteBehavior.Restrict);

            entity.HasIndex(x => new
            {
                x.DriverUserId,
                x.Status
            });
        });
    }
}
}
```

الشكل النهائي للعلاقات



وأهم نقطة في التصميم

بالنسبة لـ **MIGA**، أصبح عندنا الآن فصل واضح جدًا:

Merchant
→ يملك **Stores**

Store
→ له **StoreCategory**
→ له **ProductCategories**
→ له **Products**
→ له **Campaigns**

Customer
→ لديه **Cart**
→ يمكن أن يكون منتجًا عاديًا أو منتجًا ضمن `CartItem` Campaign

Campaign
→ `CampaignOrder` العملاء ينضمون إليها من خلال
→ `MerchantOrder` عند الوصول للعدد المطلوب يتم إنشاء
→ التاجر يجهز الطلب المجمع
→ `Delivery` بعدها يتم توزيع الطلبات الفردية على

وهذا الهيكل مناسب جدًا لفكرة **MIGA** التي شرحتها، وليس مجرد متجر إلكتروني تقليدي.

ملاحظة مهمة: في النسخة الإنتاجية التالية أنصح بإضافة `Order` عام و`OrderItem` عام فوق هذا الهيكل، لأن ذلك سيجعل MIGA يدعم لاحقًا **شراء المنتجات العادية + الشراء الجماعي + الدفع + الإلغاء والاسترجاع** بطريقة أنظف بدل توسعة `CampaignOrder` فقط.